Technical Intelligence Report: AI Dungeon Adventure Scripting Architecture, Runtime Mechanics, and Engineering Specification

Executive Technical Summary

The AI Dungeon scripting engine provides an embedded JavaScript execution environment that allows creators to intercept, parse, modify, and augment narrative data across the lifecycle of an interactive text session[1]. Operating as an event-driven interceptor pipeline situated between the player interface, the core database, and external Large Language Model (LLM) inference endpoints, the engine exposes four distinct code components: a shared Library module and three execution hooks (onInput, onModelContext, and onOutput)[1]. Execution occurs within a sandboxed virtual machine governed by strict resource ceilings: a hard execution wall of 2000 milliseconds and a heap memory cap of 16 megabytes per invocation[1]. Scripts possess mutable access to an adventure-scoped persistent storage container (state), an array of knowledge retrieval entities (storyCards), session metadata (info), and the active turn string (text)[1].

Building reliable, complex systems within this architecture introduces complex state synchronization challenges[1]. Because the underlying platform supports non-linear turn operations—such as undos, retries, direct history edits, and empty continuation prompts—naive mutations to persistent state or programmatic injections of story cards frequently suffer from state drift, turn-count desynchronization, and duplicate side effects[1]. Furthermore, the introduction of prompt caching architectures ("cache-efficient models") creates a divergence in context handling: while standard models permit arbitrary runtime rewrites of the context string sent to the model, cache-efficient models ignore programmatic alterations in the onModelContext hook to preserve pre-computed prefix caches[4]. Developing robust adventure scripts therefore requires defensive schema initialization, idempotency enforcement keyed to action counters, strict separation between deterministic script state and nondeterministic LLM prose, and modular encapsulation within the shared library[1].

Current Scripting Architecture

The AI Dungeon platform differentiates between Scenarios, which serve as reusable adventure templates, and Adventures, which are concrete, playable sessions instantiated from a scenario or started independently[1]. Understanding the boundary between these objects is fundamental to managing code deployment, state isolation, and data persistence[1].

Scenario Scripts versus Adventure Scripts

In the primary production architecture, scripts are created within and attached to Scenarios[1]. Only "Simple Start" and "Character Creator" scenario configurations permit direct script attachments through the Scenario Editor Details tab[1]. In "Multiple Choice" scenarios, scripts cannot be attached at the parent scenario level; instead, scripts must be defined independently within each individual option branch, operating in total isolation from other options[1]. Only the creator of a scenario can inspect or edit its underlying script code, though published scenarios are subject to platform moderation to ensure compliance with community guidelines[1].

When a player launches an adventure from a scripted scenario, the engine provisions a detached game session[1]. The executable JavaScript code remains linked to the parent scenario: when the scenario author edits and saves scripts in the Scenario Editor, those code modifications propagate dynamically to existing adventures instantiated from that scenario[2]. In contrast, the runtime game state (state) is strictly isolated: every adventure operates on its own dedicated instance of state, completely decoupled from the parent scenario and from any concurrent adventures started from the same template[1].

The platform architecture features an enhanced content model designated as "Adventure Scripts"[8]. This model elevates scripts into independent, discoverable platform assets that can be attached, detached, toggled, and reordered directly within an active adventure's settings menu, eliminating the requirement to restart the narrative[8]. Under this architecture, multiple independent scripts compose sequentially in an execution chain[8]. Chained scripts execute within isolated state partitions, ensuring that two independent scripts modifying properties on state do not overwrite each other's keys[8]. Furthermore, the platform enforces permission boundaries regarding system memory: while the scenario's native root script retains write access to Plot Essentials and Author's Notes, dynamically attached adventure scripts can read Plot Essentials but have their programmatic writes discarded to prevent execution conflicts[8].

Data Linkage, Cloning, and State Initialization

When an adventure is instantiated, the system performs a snapshot copy of the scenario configuration[1]. Predefined Story Cards established in the scenario editor are cloned into the adventure's private storyCards collection[1]. If the scenario contains configuration placeholders—variables declared in the scenario prompt using ${placeholder} syntax—the player's setup selections are serialized into the adventure state under state.placeholders as an array of objects containing question and answer string pairs[1].

The runtime state container (state) initializes as an empty object ({}) prior to the first script execution, unless the player completed setup placeholders, in which case state.placeholders is pre-populated[1]. Modifying state, dynamic story cards, or action history within one session has no influence on concurrent or subsequent sessions[1]. Separate adventures created from the same scenario share no mutable data at runtime[1].

Execution Lifecycle

A turn in AI Dungeon encompasses the processing steps between an initial trigger event (player input, empty continue submission, retry command, or turn undo) and the final rendering of the narrative response[1].

Turn Processing Pipeline

The execution flow of a standard narrative turn proceeds through nine sequential phases:

1. Input Submission: The player submits text through the interface, selecting an action type (Do, Say, Story, See) or triggering a Continue[1].
2. Input Normalization: The client-side engine applies initial grammatical and perspective transformations to the raw string based on the selected action mode[4].
3. onInput Hook Invocation: The engine loads the Library script, appends the Input script, and evaluates modifier(text) within the sandboxed virtual machine[3]. The parameter text contains the normalized input[1].
4. Input Interception Check: The return payload of onInput is evaluated. If the script returns { stop: true }, narrative turn progression halts immediately; the model inference engine is not invoked, and the engine transitions directly to view updates[1]. If stop is false or omitted, the modified text replaces the player input in the adventure timeline[1].
5. Knowledge Retrieval and Context Assembly: The engine scans recent history against the trigger keys of all objects in storyCards[1]. Triggered card entries are assembled alongside AI Instructions, Plot Essentials (state.memory.context), the adventure action history, Author's Notes (state.memory.authorsNote), and the current input[1].
6. onModelContext Hook Invocation: The engine prepends the Library script to the Context script and evaluates modifier(text)[3]. Here, text represents the complete assembled context payload destined for the model[1]. The script can inspect context, inspect info.storyModel, alter context text (on compatible models), or halt the loop[1].
7. LLM Generation: The finalized context prompt is transmitted across internal microservices to the designated language model backend for completion[1].
8. onOutput Hook Invocation: Upon receiving the generated completion from the LLM, the engine prepends the Library script to the Output script and evaluates modifier(text), where text contains the model's raw completion[1]. The script can redact, reformat, parse structural commands, update persistent game records in state, or alter the text[1].
9. Database Commit and View Render: The engine writes the finalized player action and AI output action to the database, updates the persistent state JSON blob, applies any dynamic changes made to storyCards, and pushes the final text to the player's viewport[1].

Turn Mutation Lifecycles

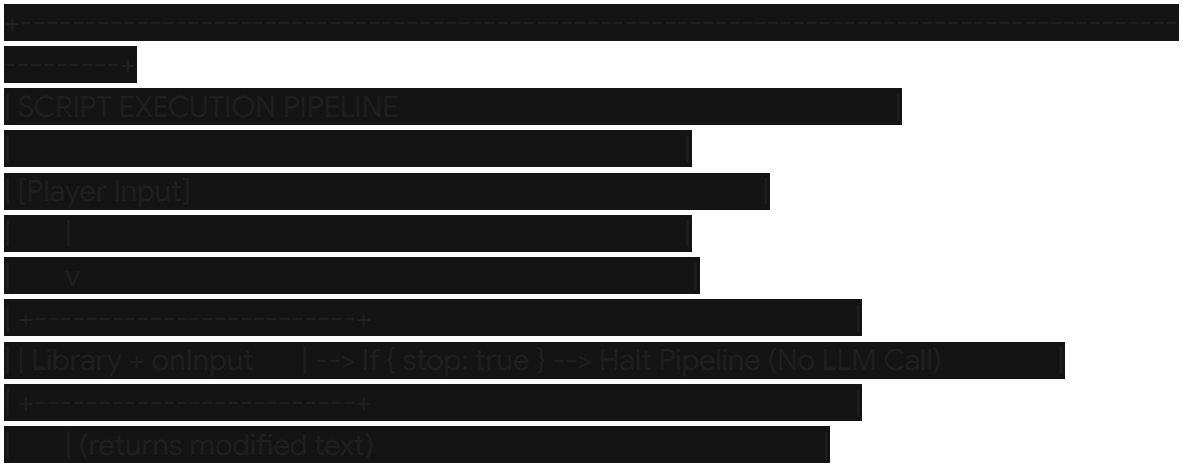
Standard linear turns represent only a subset of engine operations. Adventure scripting must reliably account for non-linear turn mutations initiated by the player:

When a player triggers a Continue by submitting an empty input field or clicking the Continue button, the onInput hook is invoked with an action type of continue and an empty or

whitespace string[1]. If not stopped by the script, execution proceeds through onModelContext, calls the LLM, and triggers onOutput to generate an extension of the existing narrative thread[1].
When a player clicks Retry to reject the latest AI output, the engine discards the most recent AI completion from the active history[11]. The onInput hook is completely bypassed because no new player input was submitted[4]. The pipeline begins immediately at Phase 5 (Knowledge Retrieval and Context Assembly), invoking onModelContext, executing model inference, and concluding with onOutput[1]. Crucially, any state mutations performed inside onOutput during the original turn remain in state unless manually rolled back; when onOutput runs again for the retry, those mutations will execute a second time if not guarded by idempotency checks[1].
When a player clicks Undo or runs /revert, the engine removes the selected player action and corresponding AI response from the action history database[11]. However, the scripting VM is not invoked during an undo operation, and the backend engine does not roll back the state JSON blob to a previous revision[1]. The persistent state object retains whatever properties were assigned during the undone turns, creating a desynchronization where info.actionCount decreases, but state remains mutated[1].
When a player directly edits the text of a previous turn in the history log, the history record is updated in place within the database[11]. No script hooks are evaluated during the edit itself[1]. On the subsequent turn, the modified text will be visible in the history array and will feed into story card keyword triggers[1].

Script-Component Reference

The AI Dungeon scripting interface segregates JavaScript code into four distinct editor tabs: Library, Input, Context, and Output[1]. Each tab possesses distinct execution scopes, parameters, return expectations, and failure conditions[1].



Library Component

The Library tab serves as a shared code repository that is prepended verbatim to the top of the Input, Context, and Output scripts prior to their evaluation[3]. It is evaluated dynamically during every hook invocation because its source code is concatenated directly before the hook's executable code[3]. The Library does not maintain a continuous shared memory heap across hooks[12]. Global variables instantiated in the Library tab during onInput cease to exist when the VM tears down; they are re-instantiated with fresh initial values during onModelContext and again during onOutput[12]. Cross-hook and cross-turn variable persistence must operate exclusively through the global state container[12].

The Library tab must not return a value or call a top-level return statement, as this breaks parsing when concatenated with subsequent hook scripts[3]. Its appropriate responsibilities include declaring helper functions, business logic algorithms, configuration parsing, dice rolling engines, state migration routines, and shared object factories[16]. It is entirely inappropriate to store volatile state in local closures expecting it to persist to the next hook, or to execute immediate mutations without knowing which hook is running[12].

```javascript
// Library Tab Pattern: Modular Namespace Declaration
var EngineCore = (function() {
  function computeSkillCheck(targetDC, attributeBonus) {
    var roll = Math.floor(Math.random() * 20) + 1;
    var total = roll + attributeBonus;
    return {
      roll: roll,
      total: total,
      success: total >= targetDC,
      critical: roll === 20
    };
  }

  function getOrInit(obj, key, defaultVal) {
    if (obj[key] === undefined) {
      obj[key] = defaultVal;
    }
    return obj[key];
  }

  return {
    computeSkillCheck: computeSkillCheck,
    getOrInit: getOrInit
  };
})();
```

Input Hook (onInput)

The Input script intercepts and processes player-submitted text prior to context assembly, story card matching, and model dispatch[1]. It runs immediately upon player submission of an action or Continue, but does not run during retries or undos[1]. Available parameters include text (the active input string), history (read-only history array), storyCards (mutable array), state (mutable persistence object), and info (session metadata)[1].

The script must evaluate the function modifier(text) as the final executable statement and return an object matching { text: string, stop?: boolean }[1]. Returning { text: modifiedText } replaces the player's submitted action with modifiedText throughout downstream processing and history recording[1]. Returning { stop: true } halts the turn pipeline immediately, bypassing the LLM[1]. If the script fails to return an object structured with the text property, the engine may

throw an unhandled script error ("Unable to run scenario scripts") or corrupt the turn input[1]. Modifiable targets include text, state, and storyCards, while modifications to history within this hook are ignored[1]. Appropriate responsibilities include command parsing (/command), player stat updates, input sanitization, dynamic trigger preparation, and blocking invalid moves[12]. It should not be used for inspecting or predicting LLM completion prose, or injecting model-facing instructions that should only be visible temporarily in context[4].

```javascript
// Input Tab Pattern: Mandatory Modifier Contract
const modifier = (text) => {
  if (!state.initialized) {
    state.initialized = true;
    state.actionWatermark = -1;
  }

  if (text.startsWith("/inspect")) {
    state.debugLogRequested = true;
    return { text: null, stop: true };
  }

  let transformedText = text.trim();
  return { text: transformedText };
};

modifier(text);
```

Context Hook (onModelContext)

The Context script intercepts the raw text prompt constructed by AI Dungeon's context builder immediately before the prompt is transmitted to the language model[1]. It runs immediately prior to LLM inference across standard turns, Continue actions, and Retries[1]. Available parameters include text (the assembled prompt string), history, storyCards, state, and info[1]. In this hook specifically, info is populated with maxChars (maximum estimated character limit for the prompt), memoryLength (character count allocated to Plot Essentials), useCacheEfficient (boolean flag denoting prompt-cache status), and storyModel (object containing model name and version)[1].

The script must conclude with modifier(text) and return { text: string, stop?: boolean }[1]. On

cache-efficient models (info.useCacheEfficient === true), the LLM backend relies on fixed prefix KV-cache storage[4]. The context builder provides the assembled text to the script for inspection, but the engine discards any modifications made to text upon return, rendering prompt-rewriting scripts ineffective[4]. On cache-efficient models, dynamic context control must be exercised via state.memory properties or through storyCards[1]. Returning { stop: true } in onModelContext aborts the inference call and renders an engine message to the user: *"Sorry, the AI is stumped. Edit/retry your previous action, or write something to help it along."*[1]. Appropriate responsibilities include context monitoring, dynamic token budgeting, character tracking verification, and context auditing[1]. Authors must avoid executing heavy business logic mutations that are not idempotent across retries[1].

```
JavaScript
// Context Tab Pattern: Context Inspection and Modification
const modifier = (text) =>
 if (info.useCacheEfficient) {
   return { text }
}

 let modifiedContext = text;
 if (state.pendingCombatEncounter)
   modifiedContext = modifiedContext + "\n[Instruction: Describe combat with brutal tactical
realism.]"
}

 return { text: modifiedContext };
}

modifier(text)
```

Output Hook (onOutput)

The Output script intercepts the raw string completion returned by the language model before it is committed to history or displayed in the user interface[1]. It runs immediately upon arrival of the LLM generation across standard turns, Continues, and Retries[1]. Available parameters include text (the AI's generated output string), history, storyCards, state, and info[1].
The script must conclude with modifier(text) and return { text: string }[1]. Returning an empty string ("") throws a critical engine exception presented to the player: *"A custom script running*

on this scenario failed. Please try again or fix the script."[1]. Returning { stop: true } in onOutput is invalid; the engine treats the literal string "stop" as the intended output text, replacing the story completion with the word "stop"[1]. Furthermore, mutations written to state.memory properties within onOutput are not factored into the active turn and only take effect on the subsequent player turn[1]. Appropriate responsibilities include redacting restricted terms, standardizing typographic formatting, extracting structured data from AI prose, updating dynamic tracking variables, and checking combat resolutions[4]. Authors must avoid invoking destructive, non-idempotent state mutations without verifying if the turn is a Retry[1].

```
JavaScript
// Output Tab Pattern: Processing Model Generations
const modifier = (text) =>
 if (!text || text.trim().length === 0)
   return { text: "The scene shifts silently..." }


 if (state.lastProcessedAction !== info.actionCount) {
   state.lastProcessedAction = info.actionCount;
   state.turnCounter = (state.turnCounter || 0) + 1
 }

 let cleanedText = text.replace(/(\n\s*){3,}/g, "\n\n")
 return { text: cleanedText }


modifier(text)
```

Runtime Environment

The AI Dungeon scripting engine executes user code within an isolated, server-side JavaScript sandbox[1]. The execution environment provides a modern ECMAScript standard library while completely disabling system, network, and persistent process interfaces[1].

The environment exposes standard ECMAScript functionality, including ES6 classes, arrow functions, object destructuring, template literals, spread syntax, standard built-ins (Object, Array, String, Math, Number, Boolean, Date), regular expressions via RegExp, and serialization via JSON[1]. However, the sandbox strips away all ambient environments typically present in browser or Node.js runtimes:

- Browser Web APIs: There is no window, document, DOM tree, Web Storage API (localStorage), or UI alert system[3].
- Node.js System APIs: The process object is inaccessible[3]. Filesystem libraries (fs), path resolvers (path), child processes (child_process), and native system bindings are strictly absent[12].
- Module Resolution: There is no module system[3]. Statements using require(), import, and export are undefined and cause immediate syntax compilation failures[3]. All modular code must be compiled or merged directly into the Library tab[3].
- Asynchronous Timers and Event Loops: The timers setTimeout, setInterval, setImmediate, and requestAnimationFrame are undefined[3]. While Promise and async/await syntax may parse syntactically, executing asynchronous delays or microtask suspensions is not supported; scripts must execute strictly synchronous code to complete within the execution window[1].
- Network Access: Network operations are blocked[12]. The primitives fetch(), XMLHttpRequest, and WebSockets are disabled[12]. Third-party services or external database connections cannot be accessed directly from script code[12].

Complete Scripting API and Exposed Global Objects

Scripts interact with AI Dungeon through an injected set of global variables and helper functions[1]. Deconstructing objects is not required; these identifiers exist directly in the global evaluation scope of each hook script[1].

```
TypeScript
interface Action {
  text: string;
  rawText: string; // Deprecated: identical to text
  type: "start" | "continue" | "do" | "say" | "story" | "see";
}

interface StoryCard {
  id: string | number;
  keys: string;
  entry: string;
  type: string;
  title?: string;
  description?: string;
```

```
interface Placeholder {
  question: string;
  answer: string;
}

interface ModelInfo {
  name: string;
  version: string;
}

interface ScriptInfo {
  actionCount: number;
  characterNames: string[];
  maxChars?: number;          // Populated strictly inside onModelContext
  memoryLength?: number;      // Populated strictly inside onModelContext
  useCacheEfficient?: boolean; // Populated strictly inside onModelContext
  storyModel?: ModelInfo;     // Populated strictly inside onModelContext
  modelName?: string;         // Populated inside onModelContext and onOutput
}
```

The global parameter inventory consists of five core objects and four helper functions:

The parameter text (string) is available across all hooks[1]. It represents the active text payload for the executing hook[1]. In onInput, it contains the player's normalized action; in onModelContext, it contains the assembled model prompt; in onOutput, it contains the raw AI completion[1]. It is immutable directly by assignment, but transformed by returning an updated { text } object from the hook's modifier function[1].

The parameter history (Array<Action>) is available across all hooks[1]. It provides an ordered, chronological array of past actions in the adventure[1]. The earliest action is at index 0 (typically type "start"), and the most recent action is at history[history.length - 1][1]. The array is read-only; in-memory mutations to the array do not persist to the database[12].

The parameter storyCards (Array<StoryCard>) is available across all hooks[1]. It contains all knowledge base cards defined in the adventure, with worldInfo maintained as a backward-compatible alias[1]. The array is fully mutable; modifications, additions, and deletions made to this array persist across turns to the database[1].

The parameter state (Record<string, any>) is available across all hooks[1]. It serves as the persistent data container serialized to the database between turns[1]. Beyond arbitrary custom fields, it contains system-reserved sub-objects:

- state.memory.context: String prepended to the absolute top of the model prompt, corresponding to the "Memory" / "Plot Essentials" field in the user interface[1].
- state.memory.authorsNote: String injected near the end of the context prompt, directly prior to the final historical action exchange[1].
- state.memory.frontMemory: String appended to the absolute end of the context prompt, after the most recent player input[1].
- state.placeholders: Read-only array of { question, answer } objects captured during scenario startup[1].
- state.message: String displayed as an in-game notification or alert to the player (implementation varies across platform versions)[1].

The parameter info (ScriptInfo) is available across all hooks with conditional fields[1]. Universal fields include actionCount (cumulative turn count) and characterNames (string array of active characters in multiplayer)[1]. Context-specific fields include maxChars, memoryLength, useCacheEfficient, and storyModel[1].

Four utility functions are injected into the runtime environment:

- addStoryCard(keys, entry, type, name, notes, options): Pushes a new Story Card object into storyCards with an automatically generated identifier[4]. Arguments define trigger keywords (keys), context injection prose (entry), optional categorization tag (type, default 'Custom'), card display title (name, default keys), and author reference notes (notes, default '')[1]. Passing { returnCard: true } in options returns the created card object; otherwise, it returns the new length of storyCards[4].
- removeStoryCard(index): Removes the card at the specified array index via storyCards.splice(index, 1), throwing an explicit Error if the index is out of bounds[4].
- updateStoryCard(index, keys, entry, type, name, notes): Overwrites properties of the card located at storyCards[index], retaining its original id and preserving existing properties if passed nullish values, throwing an Error if the index is invalid[4].
- log(...args) / console.log(...args): Serializes arguments and records them in the scenario debugger console log, accessible via the Inspect Modal, with server records expiring after 15 minutes[1].

Persistent State and Data Mechanics

The state object provides cross-turn continuity for scripted mechanics[1]. However, the persistence model relies on a strict JSON serialization boundary between the execution sandbox and the underlying database[1].

Serialization Lifecycle and Boundaries

At the conclusion of each hook's execution, the engine inspects the mutated state object[1]. Between turns, this object is serialized to a JSON string and persisted in the adventure record[1]. On the subsequent turn or hook invocation, the JSON string is parsed back into memory as a fresh JavaScript object[1].

```javascript
// Serialization Capability Reference
// Supported Data Types:
state.playerHP = 100;                    // Numbers (integer / float)
state.characterName = "Aria";            // Strings
state.inCombat = true;                   // Booleans
state.debuffs = ["poisoned", "blinded"]; // Arrays
state.questTree = { stage: 1, flags: {} }; // Nested plain objects
state.activeTarget = null;               // Null

// Unsupported / Destructive Data Types:
state.calcBonus = function() {};         // DROPPED: Functions do not serialize in JSON
state.now = new Date();                  // MUTATED: Serializes to ISO-8601 string, not Date instance
state.registry = new Map();              // MUTATED: Serializes to {} (empty object)
state.uniqueTags = new Set();            // MUTATED: Serializes to {} (empty object)
state.pattern = /regex/g;                // MUTATED: Serializes to {} (empty object)
state.unassigned = undefined;            // DROPPED: Undefined properties are omitted

// Critical Failure Mode: Circular References
state.selfRef = state;                   // CRASH: TypeError: Converting circular structure to JSON
```

The State Desynchronization Trap

The critical vulnerability in AI Dungeon script state architecture stems from how the platform handles turn rewinds[1]. When a player clicks Undo or runs /revert, the backend removes the trailing turn from the database history[11]. However, the backend does not roll back state to a snapshot matching that prior turn[1]. Similarly, when a player clicks Retry, the AI completion is discarded, but any state alterations made in the preceding onOutput hook remain committed in state[1].

If a script blindly increments a turn counter or deducts gold inside onOutput, retrying the turn will deduct gold a second time[1]. If the player hits Undo, the gold remains lost[1]. To prevent corruption, scripts must implement an Idempotent Watermark Pattern keyed to info.actionCount[1]. Checking whether savedActionCount > info.actionCount reveals that an Undo occurred, allowing the script to purge invalid forward state[1].

```
JavaScript
function synchronizeTurnState()
 var currentAction = info.actionCount

 if (!state.actionJournal)
   state.actionJournal = {}
   state.lastActionProcessed = -1


 if (currentAction <= state.lastActionProcessed) {
   for (var turn in state.actionJournal) {
     if (parseInt(turn, 10) >= currentAction)
       delete state.actionJournal[turn]


   recalculateStateFromJournal()


 state.lastActionProcessed = currentAction
```

Defensive Schema Versioning and Migration

Because live adventures remain linked to scenario scripts or can receive code updates dynamically, an updated script may encounter a state object initialized by an older revision of the code[2]. Scripts must implement explicit schema versioning and defensive property initialization[5].

```
JavaScript
function ensureStateSchema()
 var CURRENT_SCHEMA = 3

 if (state.schemaVersion === undefined)
   state.schemaVersion = 1
   state.inventory = state.inventory || [];
```

```
if (state.schemaVersion === 1) {
  state.inventory = state.inventory.map(function(item) {
    return typeof item === "string" ? { name: item, qty: 1 } : item;
  });
  state.schemaVersion = 2;
}

if (state.schemaVersion === 2) {
  state.combat = { inCombat: false, targetId: null, round: 0 };
  state.schemaVersion = CURRENT_SCHEMA;
}
}
```

Story History and Action Representation

The history parameter provides scripts with an immutable chronological record of recent interactions[1].

Action Representation and Types

The history array contains action objects representing both player moves and AI responses[1]:

- "start": The opening narrative text of the scenario, occurring exactly once at index 0[1].
- "do": A player-directed physical action submitted through Do mode, formatted by the client with leading punctuation or second-person framing ("> You...")[1].
- "say": Player-directed character speech submitted through Say mode, formatted with dialogue syntax ('> You say "..."')[1].
- "story": Direct prose input submitted by the player in Story mode, injected without character framing[1].
- "see": Descriptive sensory input generated by image-related interactions or sensory queries[1].
- "continue": Narrative text generated by the AI model, or an action record representing an empty input submitted by the player[1].

History Window Limits and Inference Caveats

Scripts cannot inspect the entire timeline of an extended adventure via history[1]. The engine exposes an array containing only recent actions—typically between 10 and 30 actions depending on memory settings and client configuration[1]. Actions beyond this boundary are truncated from history and consolidated into the game's automatic summarization and memory systems[21].

Architectural Rules for History Inference:

- Never rely on history.length as a global turn counter, as history.length plateaus once the recent window limit is reached; rely exclusively on info.actionCount for cumulative turn

indexing[1].

- Action text contains client-injected formatting, such as newline breaks (\n) and prompt angle brackets (>); string matching and regular expressions must sanitize leading characters and punctuation when scanning action history[4].
- Edits made by the player to past actions take effect retroactively in the history array on subsequent turns[11].

AI Context Construction and Prompt Assembly

Context assembly transforms stored story elements, player actions, and script modifications into the unified text prompt passed to the LLM[1].

Prompt Assembly Hierarchy

Context is constructed from top to bottom according to a strict priority hierarchy[1]:

1. AI Instructions: Global system prompts defining persona, narrative constraints, tone, and prose rules (position 1 in context)[13].
2. Plot Essentials (Memory): Fundamental world facts and high-level plot constraints stored in state.memory.context (or the Memory UI)[1].
3. World Lore (Triggered Story Cards): Text entries from storyCards whose keywords match terms appearing in recent actions[1].
4. Story History: Chronological historical actions (start, do, say, continue), truncated from the top (oldest actions dropped first) to fit within token limits[1].
5. Author's Note: Style directives and immediate focus guidance stored in state.memory.authorsNote, placed directly before recent story interactions[1].
6. Recent Action / Current Input: The player's active command or latest continuation prompt[1].
7. Front Memory: Text defined in state.memory.frontMemory, injected at the absolute end of the context prompt, following the current input[1].

Token Budgeting, Truncation, and Estimation

The total character length of the assembled context is bounded by info.maxChars, which provides an estimated character ceiling derived from the underlying LLM's context window token limit[1]. When the cumulative character count exceeds info.maxChars, the engine executes prioritized truncation[1]. Fixed components (Instructions, Plot Essentials, Front Memory, Author's Note) are protected from partial truncation[1]. Story Cards are prioritized by keyword relevance; if cards consume excessive space, lower-priority cards fail to load[1]. Narrative History absorbs the truncation deficit: older turns are systematically dropped until the prompt fits within info.maxChars[12]. Consequently, injecting massive strings into state.memory.context or pushing excessive text into Front Memory directly displaces story history, accelerating narrative amnesia[4].

On cache-efficient models (info.useCacheEfficient === true), the LLM backend pre-computes and caches the prompt prefix (AI Instructions, Core Memory, and early Story History) in GPU

memory[4]. In this environment, the onModelContext hook can still inspect the assembled context string via text, but returned mutations are ignored[4]. On cache-efficient models, dynamic context control must be exercised via state.memory properties or through storyCards[1].

Story Cards Interaction

Story Cards represent AI Dungeon's built-in retrieval-augmented knowledge base[1].

Schema, Properties, and Activation

Every card object within the storyCards array adheres to a standard schema: id (unique identifier), keys (comma-separated trigger keywords), entry (text injected into context upon matching), type (categorical label such as 'Character', 'Location', 'Faction', 'Item', or 'Custom'), title (display label), and description (out-of-character reference notes)[1].

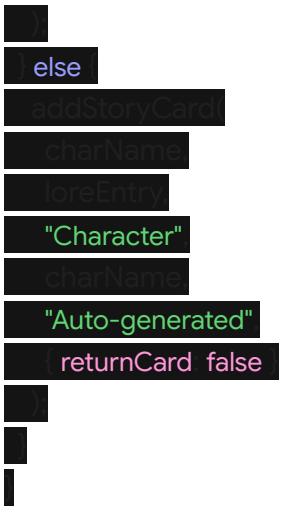
The engine triggers a story card when any comma-delimited keyword in keys appears within the search window of recent narrative text[4]. Matching is case-insensitive and operates on substring overlap rather than strict token boundaries[19]. For example, the trigger key "cat" will fire on "catch", "caterpillar", and "scat"[19]. Trigger keys must be carefully designed to prevent false positives (e.g., " Sir Arthur ", "Arthur,", or distinct full names)[19]. Content within description or notes is never sent to the LLM; it is strictly accessible to human players and scripts[4].

Programmatic Story Card Management

Scripts can dynamically construct, inspect, mutate, and delete story cards using both native array methods and injected utility functions[4].

```
JavaScript
function upsertCharacterCard(charName, loreEntry)
 var cardIndex = storyCards.findIndex(function(card)
  return card.title === charName || card.keys.split(",").includes(charName)
});

 if (cardIndex !== -1)
  updateStoryCard(
   cardIndex,
   charName,
   loreEntry,
   storyCards[cardIndex].type,
   charName,
   "Updated via Script at turn " + info.actionCount
```



Advanced scripts also utilize Story Cards as human-readable configuration stores[17]. Scripts such as *Auto-Cards* and *Inner Self* create dedicated configuration cards (e.g., title: "Configure Auto-Cards", keys: "config_autocards") with entries containing structured key-value lines[17]. Players can adjust script settings directly from the in-game Story Card UI without touching JavaScript code, and scripts parse these parameters during onInput[5].

Model Interaction and Behavioral Nuance

Interactive scripts operate alongside diverse LLM backends[4]. A critical engineering objective is separating deterministic JavaScript script behavior from model-dependent generative behavior[1].

AI Dungeon deploys multiple LLM families across subscription tiers, including Fable (Lunaris), DeepSeek V4 Flash and Pro, Gemma 4, GLM, and enterprise-grade models like Claude[6]. Scripts can inspect model parameters dynamically in the onModelContext and onOutput hooks via info.storyModel (containing { name: string, version: string }), info.modelName, and info.useCacheEfficient[4].

Because users can switch active models mid-adventure via the gameplay settings menu, script logic must avoid hardcoded model assumptions[4]. Instead, scripts should dynamically evaluate info.useCacheEfficient and info.maxChars on every turn to adapt context assembly strategies accordingly[1].

Input Processing

The onInput hook provides authority to inspect, sanitize, enrich, or intercept player commands before they are processed by the game engine[1].

By evaluating player commands and returning { stop: true }, scripts can implement custom command systems without consuming model generation tokens or incurring network latency[1].

```JavaScript
// Slash-Command Interceptor Pattern in onInput
const modifier = (text) =>
 var trimmed = text.trim();

 if (trimmed.startsWith("/"))
  var parts = trimmed.slice(1).split(" ");
  var command = parts[0].toLowerCase();
  var args = parts.slice(1);

  if (command === "stats")
   var charStats = state.stats || { strength: 10, agility: 10, intelligence: 10 };
   var report = "=== CHARACTER STATS ===\n" +
                "STR: " + charStats.strength + " | " +
                "AGI: " + charStats.agility + " | " +
                "INT: " + charStats.intelligence;

   state.message = report;
   return { text: null, stop: true };

  if (command === "sethp")
   var amount = parseInt(args[0], 10);
   if (!isNaN(amount))
    state.hp = amount;
    state.message = "Health updated to " + amount;
   return { text: null, stop: true };

 return { text };

modifier(text);
```

Scripts can also scan natural language inputs for specific trigger verbs (e.g., "try to", "attempt to", "attack") and append mechanical resolutions directly to the player's text[4].

```
JavaScript
// Dynamic Action Resolution Injection in onInput
const modifier = (text) =>
 var attemptRegex = /^(?:you\s+)?(try\s+to|attempt\s+to)\s+(.+)/i
 var match = attemptRegex.exec(text.trim())

 if (match) {
  var roll = Math.floor(Math.random() * 20) + 1
  var outcome = roll >= 12 ? "SUCCESS" : "DIFFICULTY ENCOUNTERED"
  var enrichedInput = text.trim() + " [Mechanic Resolution: " + outcome + " (Roll " + roll + "/20)]"
  return { text: enrichedInput }
 }

 return { text }
}

modifier(text)
```

Output Processing

The onOutput hook evaluates raw completions returned by the LLM, operating as both a narrative sanitizer and an event-extraction pipeline[1].

LLMs frequently develop repetitive habits, echo prompt instructions, or insert malformed line breaks[4]. Output scripts can apply regex sanitizers to normalize narrative formatting[4].

```
JavaScript
// Output Formatting and Cleaning in onOutput
const modifier = (text) =>
 if (!text) return { text: "..." }

 var cleaned = text;
 cleaned = cleaned.replace(/\[Instruction:[^\]]*\]/gi, "");
 cleaned = cleaned.replace(/\[Author'?s Note:[^\]]*\]/gi, "");
 cleaned = cleaned.replace(/\n{3,}/g, "\n\n");
 cleaned = cleaned.replace(/You says,/g, "You say,");
```

```
return { text: cleaned };
};

modifier(text);
```

To allow the LLM to trigger game mechanics (such as dealing damage or awarding items), prompt instructions can direct the model to emit bracketed metadata tags (e.g., [GOLD: +15], [DAMAGE: 4])[5]. The Output script extracts these tags via regular expressions, applies the corresponding mutations to state, and strips the tags from the prose before display[4].

```
JavaScript
// Structural Event Extraction in onOutput
const modifier = (text) => {
  var outputText = text;
  var itemRegex = /\[ITEM:\s*([^\]]+)\]/gi;
  var match;

  while ((match = itemRegex.exec(outputText)) !== null) {
    var itemName = match[1].trim();
    if (!state.inventory) state.inventory = [];
    state.inventory.push({ name: itemName, acquiredTurn: info.actionCount });
  }

  outputText = outputText.replace(itemRegex, "");
  return { text: outputText };
};

modifier(text);
```

Library Architecture

Because the Library tab is concatenated above each modifier script, code organization within the Library determines the maintainability and scope safety of the entire project[3].
To avoid variable collisions, pollution of the global execution scope, and catastrophic re-declarations, complex scripts should encapsulate sub-systems within Immediately Invoked Function Expressions (IIFEs) or explicit namespace containers[17].

```javascript
// Library Architecture: Unified Namespace Pattern
var App = (function() {
  var VERSION = "2.1.0";

  var Database = {
    getCard: function(predicate) {
      return state.storyCards.find(predicate);
    },
    ensureCard: function(title, keys, entry, type) {
      var card = this.getCard(function(c) { return c.title === title; });
      if (!card) {
        addStoryCard(keys, entry, type || "Custom", title);
      }
    }
  };

  var Combat = {
    processTurn: function(rollMod) {
      var dc = 12;
      return Math.floor(Math.random() * 20) + 1 + rollMod >= dc;
    }
  };

  return {
    version: VERSION,
    Database: Database,
    Combat: Combat
  };
})();
```

A frequent error in script development is declaring stateful variables in the Library tab and expecting them to persist across hooks[12]. Local variables declared in the Library are reset every time a hook executes[12]. Shared variables that must survive across hooks (onInput to onModelContext to onOutput) or across turns must be stored directly on the state global container[12].

Script Execution Limits

Operating within the shared infrastructure of AI Dungeon requires strict adherence to system

resource limits[1]. The execution sandbox enforces hard boundaries that terminate runaway scripts to preserve backend stability[1].

Metric	Hard System Limit	Practical Target	Failure Consequence	Source Quality
Execution Timeout	2000 milliseconds[1]	< 100 milliseconds	Turn aborted; unhandled exception alert[1]	Verified / Documented[1]
Heap Memory Ceiling	16 Megabytes[1]	< 4 Megabytes	Immediate VM crash; state commit failure[1]	Verified / Documented[1]
state Serialization	Unspecified	< 250 Kilobytes	Slow turn latency; truncation or database reject	Inferred from Community[1]
Context Character Cap	Model-specific (info.maxChars)[1]	Adhere to info.maxChars	Automatic truncation of story history[1]	Verified / Documented[1]
Server Log Retention	15 Minutes[1]	N/A	Logs purged from Inspect modal[1]	Verified / Documented[1]

Errors and Failure Behavior

When a script violates execution boundaries or contains runtime errors, the platform handles the failure according to the active stage[1]:

- Syntax Compilation Failure: If any script contains a syntax error (e.g., unclosed brackets, invalid tokens), the turn fails instantly, and the UI displays an alert: *"Unable to run scenario scripts."*[1].
- Execution Timeout: If code executes an infinite loop or excessive computation exceeding 2000 ms, the server terminates the process and marks the turn as failed[1].
- Empty String Return in onOutput: Throws a visible script failure: *"A custom script running on this scenario failed. Please try again or fix the script."*[1].

- Malformed Return Objects: Omitting the return object in modifier causes the hook to evaluate as undefined, resulting in corrupted narrative injection or engine errors[1].

Debugging and Observability

The Scenario Scripting UI includes dedicated diagnostic panels to monitor runtime execution[3]:

- Script Test Panel: Located on the right side of the Script Editor, this panel allows developers to inject synthetic test strings into text, state, and storyCards, run the hook locally, and inspect the resulting output payload, state mutations, and generated console logs[1].
- Inspect Modal (Brain Icon): Accessible from the navigation bar while editing scripts, this modal provides two monitoring views:
  - Last Model Input: Displays the complete assembled context prompt as transmitted to the LLM during the author's most recent playtest adventure[12].
  - Script Logs & Errors: Displays timestamped console logs emitted by log() or console.log() across onInput, onModelContext, and onOutput (retained for 15 minutes)[1].

Because console.log output requires switching windows to the Scenario Editor Inspect modal, developers frequently create a self-contained "Debug Card" within storyCards[24]. By writing internal state reports or error traces directly to the entry of a dedicated Story Card (e.g., titled "[DEBUG LOG]" with keys set to a non-matching string like "zxzx_debug"), authors can inspect real-time state mutations directly from the in-game Story Cards menu during playtesting[24].

Testing Methodology

Developing adventure scripts requires a multi-stage testing methodology that bridges offline unit tests and in-engine runtime validation[1]. Conventional Node.js unit tests often produce false passes because standard Node environments expose globals (setTimeout, process, fetch) and module systems that do not exist in AI Dungeon's sandbox[12].

Offline testing must utilize a custom harness that mocks the AI Dungeon runtime:

1. Concatenate Library.js with the target hook script (onInput.js, onModelContext.js, or onOutput.js)[3].
2. Execute the concatenated source inside a sandboxed VM context with all Node globals stripped[1].
3. Inject mock globals: text, history, storyCards, state, info, and utility functions (addStoryCard, removeStoryCard, updateStoryCard, log)[1].
4. Assert that the returned object contains { text }, verifies schema changes in state, and checks that execution completes well within 2000 ms[1].

In-engine testing must specifically validate non-linear edge cases:

- Retry Verification: Submit an action, trigger Retry three times consecutively, and assert that state counters or inventory additions do not multiply[1].
- Undo Verification: Advance three turns, execute two Undos, submit a new action, and verify that internal state rolled back to match the current turn[1].

- Continue Verification: Submit an empty Continue prompt and confirm that parsing logic does not crash on empty strings[1].
- Model Switch Verification: Switch between standard and cache-efficient models in settings, verifying that context injection falls back gracefully to storyCards[4].

Deployment and Maintenance Workflow

Installing, operating, and maintaining scripts follows a structured workflow across AI Dungeon's interface[2].

To install a script into a scenario:

1. Navigate to the AI Dungeon website on desktop (script editing is restricted on mobile clients)[2].
2. Create a new Scenario or edit an existing scenario[2].
3. Open the Details tab, scroll down to Scripting, and toggle Scripts Enabled to ON[2].
4. Select Edit Scripts to open the multi-tab scripting editor[2].
5. Paste corresponding code into the Library, Input, Context, and Output tabs[2].
6. Click the yellow Save button in the upper navigation bar[2].

When testing and maintaining deployed scripts:

- Changes saved to a scenario's scripts apply immediately to all newly instantiated adventures, and propagate dynamically to existing adventures derived from that scenario[2].
- Under the Adventure Scripts architecture, standalone scripts can be enabled or reordered directly from an adventure's in-game settings under the Scripts menu[8].
- Creators should maintain code backups locally or in Git repositories, as the in-app script editor does not maintain version control or revision rollback history[2].

Analysis of Existing Sophisticated Scripts

Examining established community architectures highlights how experienced developers overcome platform limitations:

Auto-Cards (developed by LewdLeah) automates worldbuilding continuity by generating dynamic Story Cards during play[23]. In onInput, it scans narrative text for recurring capitalized proper nouns[2]. In onModelContext, it appends an instruction prompting the AI to describe novel entities in a structured format[4]. In onOutput, it extracts the generated profile, invokes addStoryCard(), and strips the generation metadata from the player's visible story text[4].

Inner Self (developed by LewdLeah) implements persistent NPC psychology, goals, and internal monologue[16]. The script uses Story Cards as external memory banks for NPCs[17]. During onModelContext, it detects active characters in the scene, pulls their private thoughts from their corresponding story cards, and injects directives instructing the model to reflect those hidden motivations in character dialogue and body language[4].

Story Arc Engine (developed by Yi1i1i) manages macro-level narrative pacing across extended campaigns[2]. At set turn intervals, the script directs the LLM to outline high-level plot goals and

three-act narrative milestones, storing the output in state.memory.authorsNote to keep future completions aligned with long-term objectives[2].

Hashtag-DnD (developed by raeleus) introduces a tabletop RPG framework using # command routing[24]. The script maintains character stats, inventories, spellbooks, and dice calculations inside state, injecting combat summaries into context while cleaning output prose to ensure smooth narrative flow[25].

Common Failure Modes

The following failure catalog outlines recurring platform hazards and their engineering mitigations:

Turn Desynchronization via Undo

- Cause: The player executes an Undo or /revert, rewinding history and info.actionCount, but the engine leaves state un-reverted[1].
- Symptoms: Character stats, quest flags, or inventories retain forward progress from turns that were undone[1].
- Detection: In onInput, check if state.lastActionProcessed >= info.actionCount[1].
- Prevention: Implement an idempotent action watermark and maintain a historical state log to allow programmatic rollbacks[1].
- Recovery: Prune journal entries exceeding info.actionCount and re-derive current state from the remaining log[1].

Duplicate Mutations via Retry

- Cause: The player triggers Retry; onModelContext and onOutput run again without an intervening onInput[4].
- Symptoms: Resource counters decrement twice, experience awards double, or inventory items duplicate on a single turn[1].
- Detection: Compare state.lastOutputAction against info.actionCount inside onOutput[1].
- Prevention: Guard all state mutations inside onOutput behind if (state.lastOutputAction !== info.actionCount)[1].
- Recovery: Revert uncommitted turn mutations if an identical actionCount is encountered[1].

Cache-Efficient Context Overwrite Failure

- Cause: Script attempts to modify prompt text by returning { text: modified } from onModelContext while running on a cache-efficient model[4].
- Symptoms: Injected instructions or custom prompt modifications are ignored by the LLM[4].
- Detection: Inspect info.useCacheEfficient === true inside onModelContext[4].
- Prevention: Route dynamic context updates through state.memory properties or storyCards rather than modifying text directly[1].
- Recovery: Fall back dynamically to Story Card updates when cache efficiency is

detected[4].

False-Positive Story Card Activation

- Cause: Defining short or generic words as triggers in storyCards[i].keys (e.g., "in", "cat", "dan")[19].
- Symptoms: Irrelevant story cards trigger continuously on common syllables, bloating context and displacing narrative history[4].
- Detection: Inspect the Last Model Input view in the Inspect modal to check which cards fired[12].
- Prevention: Use specific, multi-token keywords or pad trigger words with spaces (e.g., " Dan ", "Dan,")[19].
- Recovery: Implement an initialization audit that scans storyCards and warns or sanitizes broad keys[4].

Circular Reference Serialization Crash

- Cause: Attaching an object reference to state that directly or indirectly references itself[1].
- Symptoms: Script engine crashes with an unhandled exception upon turn completion; the turn fails to commit[1].
- Detection: Execute try { JSON.stringify(state); } catch(e) { log(e); } during development.
- Prevention: Store only plain arrays, primitive values, and hierarchical object trees in state[1].
- Recovery: Implement a defensive object cloner that strips circular pointers prior to hook completion[1].

Platform Changes and Version Drift

AI Dungeon has undergone several major architectural transitions that impact script behavior and compatibility[7].

During the legacy platform era (2019–2021), scripts interacted with OpenAI GPT-3 backends[12]. Context modifications were constrained by a strict 75% edit distance ceiling enforced by external safety policies[12]. Knowledge base entities were termed "World Info," managed through addWorldEntry(), updateWorldEntry(), and removeWorldEntry()[12]. An undocumented flag isNotHidden existed on world entries[12].

The Phoenix platform overhaul (2023) redesigned the core interface and backend microservices, introducing the four-tab Scripting UI (Library, Input, Context, Output)[1]. World Info was rebranded to Story Cards, and addStoryCard(), updateStoryCard(), and removeStoryCard() became the primary APIs[1].

Subsequent platform enhancements (2024–2026) introduced doubled context windows, automatic memory summarization, cache-efficient prompt caching, and metadata expansion (info.useCacheEfficient, info.storyModel, info.modelName)[4]. The rollout of the Adventure Scripts architecture elevated scripts to standalone, composable entities with private state partitioning and Plot Essentials write protection[8].

Deprecated APIs and obsolete patterns that should not be used in modern script development

include:

- Legacy World Info APIs: Avoid addWorldEntry, removeWorldEntry, and updateWorldEntry; use modern StoryCard functions[1].
- Action Raw Text: Avoid action.rawText; use action.text[1].
- Built-in RPG Attributes: Properties such as state.displayStats, state.skills, state.inventory, and state.stats no longer link to native UI widgets[12].
- Script UI Alerts: state.message displays inconsistently across platforms and should not be relied upon for critical game mechanics[1].

Undocumented and Implementation-Defined Behaviors

Several critical operational behaviors lack formal documentation and have been identified through empirical analysis and community testing:

The Library tab's integration relies on physical source code concatenation[12]. Rather than compiling the Library as an isolated module, the engine executes compiledScript = libraryCode + "\n" + hookCode;[12]. Consequently, a missing closing bracket, syntax error, or unclosed multi-line comment in the Library tab will simultaneously corrupt all three execution hooks[3].

Mutations to state.memory properties (context, authorsNote, frontMemory) within the onOutput hook exhibit a deferred execution delay[1]. Because context assembly for the active turn has already completed, changes written to memory inside onOutput do not affect the turn just generated; they are committed to the model prompt only on the subsequent player turn[1].

Returning { stop: true } in onInput halts turn processing, but in specific client revisions, it can prevent the player's input text from appearing in the visible adventure history[12]. Furthermore, while state serializes deep JSON hierarchies, prototype chains are completely discarded between turns; instances of custom ES6 classes deserialize as plain Object instances on the subsequent turn[1].

Security, Safety, and Computational Efficiency

Executing JavaScript inside an embedded sandbox requires defensive programming against performance bottlenecks and memory exhaustion[1].

To prevent Regular Expression Denial of Service (ReDoS) and CPU timeouts (2000 ms limit), regular expressions executed over narrative text must avoid nested quantifiers such as (a+)+ or unbounded capture groups[1]. Authors should anchor pattern matches and enforce explicit length bounds (e.g., /.{1,100}/)[26].

Context efficiency requires careful token management[4]. Every character injected into state.memory.context, frontMemory, or triggered Story Cards reduces the token budget available for narrative history[1]. Scripts should inject compressed, natural-language state summaries rather than serializing raw JSON objects into model-visible context[13]. When inspecting past actions in onInput or onOutput, scripts should restrict string scanning to the

last 3–5 actions rather than iterating across the entire history array[5].

Reusable Architectural Design Patterns

Robust adventure scripts rely on established design patterns tailored to AI Dungeon's constraints:

Centralized Idempotent State Journal: To survive undos and retries, scripts should record turn mutations in an append-only journal indexed by info.actionCount[1]. When an undo is detected (currentAction < lastProcessedAction), the journal discards future turn entries and recalculates the active state from historical checkpoints[1].

Story Card Configuration Interface: To allow non-technical players to configure scripts without modifying JavaScript code, scripts can initialize a dedicated configuration Story Card[17]. In onInput, the script locates the card, parses user-edited key-value pairs (e.g., Difficulty: Hard), and updates internal mechanics dynamically[5].

Bracketed Event Parsing: To extract structured actions from the LLM, the script prompts the model to output metadata inside brackets (e.g., [HP: -10])[5]. In onOutput, regular expressions parse and remove these tags before the text is displayed to the player, maintaining clean prose while tracking mechanics deterministically[4].

The Boundary Between Deterministic Code and Nondeterministic LLM Output

Adventure scripts operate at the boundary between deterministic programmatic rules and probabilistic language generation[1]. Maintaining narrative coherence requires clear architectural boundaries[5]:

Deterministic script state must always remain authoritative over mechanical facts[5]. If an LLM completion narrates that a player acquired an item, but the script's inventory validation logic rejects the acquisition, the script must redact or correct the narrative completion in onOutput[4]. Scripts should never assume the LLM will follow formatting instructions with 100% fidelity; regex parsers must accommodate missing delimiters, irregular casing, and unexpected whitespace[5].

To prevent narrative feedback loops, instructions injected into context must be scrubbed from the final story text[4]. If an instruction tag or author note accidentally leaks into the visible adventure history, the model will imitate the artifact on subsequent turns, degrading prose quality[4].

Coding-Agent Engineering Handoff

This specification provides operational rules and checklists for autonomous agents developing or maintaining AI Dungeon Adventure Scripts.

Known Platform Guarantees

- Lifecycle Sequence: Execution hooks run in the strict order onInput -> onModelContext

-> onOutput during standard player actions[1].

- State Persistence: Primitive values, plain arrays, and plain objects attached to state persist across turns via JSON serialization[1].
- Knowledge Base Mutation: Modifications made to storyCards via addStoryCard(), updateStoryCard(), or native array methods persist to the adventure database[1].
- Pipeline Interception: Returning { stop: true } in onInput halts the execution pipeline before LLM generation occurs[1].

Defensive Engineering Assumptions

- Turn Mutation Safety: Always assume the player will use Undo or Retry[1]. Never execute non-idempotent state mutations without checking info.actionCount watermarks[1].
- Cache Invalidation: Assume models may utilize prompt caching[4]. Never rely on modifying text in onModelContext for core game mechanics; route context through storyCards or state.memory[1].
- Volatile Local Scope: Assume all variables declared outside of state inside the Library tab will be reset between hooks[12].
- Schema Evolution: Assume state may arrive with missing keys or legacy schemas; execute defensive initialization on every hook entry[5].

Live Testing Requirements

Before deploying a script, verify the following in the live AI Dungeon testing harness:

- Confirm that submitting an empty Continue action does not throw parsing errors in onInput[1].
- Confirm that executing three consecutive Retries does not duplicate state mutations or inventory additions[1].
- Confirm that executing an Undo decrements info.actionCount and triggers internal rollback handlers correctly[1].
- Confirm that total context character length remains within info.maxChars to prevent history truncation[1].

Forbidden Anti-Patterns

- Do not invoke asynchronous timers (setTimeout, setInterval) or expect microtasks to delay execution[3].
- Do not attempt network operations (fetch, http) or filesystem calls (fs)[12].
- Do not return raw strings, null, or empty objects from modifier; adhere to the { text, stop } interface[1].
- Do not return an empty string ("") in onOutput[1].
- Do not attach circular object references or ES6 class instances to state[1].

Pre-Coding Architecture Checklist

1. Modularize shared code inside the Library tab using an IIFE structure[17].
2. Define a schema version number (state.schemaVersion = 1) and a migration pipeline[5].

3. Implement an action watermark using info.actionCount to guard against retry duplication and undo desynchronization[1].
4. Choose the appropriate context injection mechanism: use storyCards or state.memory to ensure compatibility with cache-efficient models[1].
5. Define explicit command prefixes (e.g., /command) and return { text: null, stop: true } on command execution[1].

Pre-Deployment Verification Checklist

1. Verify that every non-library tab concludes with modifier(text);[2].
2. Wrap all JSON parsing and regex evaluations in defensive try/catch blocks[1].
3. Audit console logging to avoid filling the 15-minute server buffer[1].
4. Inspect the Last Model Input view in the Inspect modal to ensure core story history is not being displaced[12].
5. Confirm that JSON.stringify(state) executes without throwing serialization errors[1].

Machine-Usable Technical Reference

Script Stages

Stage	Executes When	Receives	Can Modify	Must Return	Persistent Effects	Important Hazards
Library	Pre-pended before each hook evaluation[3]	None[3]	Declarations only[3]	Nothing (declarations)[3]	None (ephemeral scope)[12]	Syntax errors break all stages; variables do not persist across hooks[3].
onInput	Player submits action (Do, Say, Story, See, Continue	text: player input string[1]	text, state, storyCards [cite: 1]	{ text: string, stop?: boolean } [cite: 1, 2, 12]	Modified text is committed to history database[1]	Returning stop may hide player input; unhandled errors

	)[1]					abort turn[3].
onModelContext	Context builder completes prompt before LLM call[1]	text: assembled model prompt[1]	text, state, storyCards [cite: 1, 4]	{ text: string, stop?: boolean } [cite: 1, 2]	None directly (transient context payload)[4]	Text changes ignored on cache-efficient models[4]; stop throws AI error[1].
onOutput	LLM returns raw generation[1]	text: AI generation string[1]	text, state, storyCards [cite: 1, 4]	{ text: string } [cite: 1, 2, 3]	Modified text is saved to history and rendered[1]	Returning empty string crashes turn[1]; retries cause duplicate state updates[1].

Runtime Objects and APIs

Identifier	Type	Scope Availability	Persistence Semantics	Documented	Core Purpose / Caveats
text	string	Input, Context, Output[1]	Hook transient; transformed via return[1]	Yes[1]	Represents active turn string at each respective lifecycle phase[1].

history	Array<Action>	All hooks[1]	Read-only; backed by adventure database[12]	Yes[1]	Chronological array of recent actions ({ text, rawText, type })[1].
storyCards	Array<StoryCard>	All hooks[1]	Fully mutable; persisted across turns[1]	Yes[1]	Knowledge base entries triggered by keyword matching[1]. Alias: worldInfo[1].
state	Object	All hooks[1]	Fully mutable; serialized to JSON across turns[1]	Yes[1]	Main game state container; does not roll back on undo automatically[1].
state.memory	Object	All hooks[1]	Mutable; structured context injection[1]	Yes[1]	Contains context, authorsNote, and frontMemory[1]. Output edits defer 1 turn[1].
state.placeholders	Array<Object>	All hooks[1]	Read-only; populated at adventure start[1]	Yes[1]	Captures scenario setup answers ({ question,

					answer })[1].
info	Object	All hooks[1]	Read-only session metadata[1]	Yes[1]	Contains actionCount, characterNames[1]. Context hook adds maxChars, useCacheEfficient[1].
addStoryCard()	Function	All hooks[4]	Mutates storyCards array[4]	Yes[4]	Helper to push new story card with generated ID[4].
removeStoryCard()	Function	All hooks[4]	Splices storyCards array[4]	Yes[4]	Helper to remove card at index; throws on invalid index[4].
updateStoryCard()	Function	All hooks[4]	Mutates card at index[4]	Yes[4]	Helper to overwrite card properties; preserves original ID[4].
log()	Function	All hooks[4]	Ephemeral; server buffer expires in 15 mins[1]	Yes[4]	Diagnostic logging to Scenario Inspect modal[3].

					Alias: console.log ()[12].

Persistent State Mechanics

Behavior	Result	Confidence	Evidence
JSON Serialization	Objects, arrays, strings, numbers, booleans, null persist[1]	Verified / Documented[1]	Platform documentation and working implementations[1].
Prototype Stripping	Custom class instances deserialize as plain Object [cite: 1]	Verified in Implementation[1]	Standard JSON serialization boundary across sandbox invocations[1].
Function Dropping	Functions attached to state are omitted upon save[1]	Verified in Implementation[1]	Standard JSON stringify behavior in V8 engine[1].
Circular Reference	Throws TypeError; causes turn execution failure[1]	Verified in Implementation[1]	Standard V8 JSON serialization failure modes[1].
Undo Handling	state is NOT rolled back when turns are undone[1]	Verified in Implementation[1]	Official scripting documentation and community reports[1].
Retry Handling	state retains mutations from rejected generation[1]	Verified in Implementation[1]	Official scripting documentation and community reports[1].

Story and Context Systems

System	Script Access	Persistence	Model Visibility	Token Impact	Caveats
AI Instructions	Read-only in Context[4]	Scenario-level[13]	Position #1 in context prompt[13]	Fixed allocation[13]	Sits at top of prompt; cannot be modified by scripts[13].
Plot Essentials	state.memory.context [cite: 1]	Adventure-level[1]	Prepended before history[1]	Protected allocation[1]	Output hook modifications defer to subsequent turn[1].
Story Cards	storyCards array[1]	Adventure-level[1]	Injected when keywords match[1]	Consumes context on trigger[1]	Substring matching can cause false-positive activations[1][9].
Story History	history array[1]	Database history[1]	Middle context block[1]	Truncated when budget exceeded[12]	Truncated from oldest actions first; bounded array size[1].
Author's Note	state.memory.authorsNote [cite: 1]	Adventure-level[1]	Injected near end of context[1]	Fixed allocation[1]	Strongly steers short-term narrative tone and style[2].
Front	state.memo	Adventure-l	Absolute	Directly	Incomplete

Memory	ry.frontMemory [cite: 1]	evel[1]	end of prompt[1]	steers immediate completion[1]	sentences cause model to complete prompt fragment[12].

User Action Types

Action Type	Script Representation	Input Hook Runs?	Output Hook Runs?	Action Count Impact	Retry / Undo Implications
Do	type: "do" [cite: 1]	Yes[4]	Yes[4]	Increments by 1[1]	Standard forward progression[1].
Say	type: "say" [cite: 1]	Yes[4]	Yes[4]	Increments by 1[1]	Standard forward progression[1].
Story	type: "story" [cite: 1]	Yes[4]	Yes[4]	Increments by 1[1]	Standard forward progression[1].
See	type: "see" [cite: 1]	Yes[4]	Yes[4]	Increments by 1[1]	Standard forward progression[1].
Continue	type: "continue" [cite: 1]	Yes[4]	Yes[4]	Increments by 1[1]	Handled as standard turn with empty input[1].

Retry	Bypassed[4]	No[4]	Yes[4]	Unchanged[1]	Output hook runs again; requires idempotency guard[1].
Undo / Revert	Bypassed[1]	No[1]	No[1]	Decrements by 1+[1]	State does not roll back automatically[1].
Action Edit	Bypassed[1]	No[1]	No[1]	Unchanged[1]	Changes appear in history on next turn[11].

Runtime Limits

Resource	Documented Limit	Observed Practical Limit	Source	Confidence
Execution Time	2000 milliseconds[1]	< 100 milliseconds	Official API Guide[1]	Verified / Documented[1]
Heap Memory	16 Megabytes[1]	< 4 Megabytes	Official API Guide[1]	Verified / Documented[1]
Persistent State Size	Unspecified	~250–500 Kilobytes	Community Repositories[1]	Inferred from Implementation[1]
Context Character Length	Model-dependent (info.maxChars)[1]	Strict adherence to info.maxChars	Official API Guide[1]	Verified / Documented[1]

Server Log Retention	15 Minutes[1]	15 Minutes	Official UI Guide[1]	Verified / Documented[1]

Platform Compatibility and Version History

Feature / Interface	Current Status	Historical Predecessor	Operational Migration Concern
Story Card Helpers	Standard (addStoryCard, etc.)[1]	addWorldEntry, removeWorldEntry [cite: 12]	Legacy functions are deprecated; replace with StoryCard APIs[1].
Card Collection	storyCards [cite: 1]	worldInfo [cite: 1, 12]	worldInfo remains supported as backward-compatible alias[1].
Action Text	action.text [cite: 1]	action.rawText [cite: 1]	rawText is deprecated; standardize on action.text[1].
Built-in RPG System	Deprecated / Inactive[12]	state.stats, state.inventory [cite: 12]	Properties no longer bind to UI; build custom trackers[12].
Context Modification	Cache-dependent[4]	Unrestricted context editing[4]	Context rewrites fail on cache-efficient models; use Story Cards[4].
Script Architecture	Adventure Scripts (Alpha)[8]	Scenario Scripts only[1]	Standalone scripts run chained with private state partitions[8].

Unresolved Platform Behaviors

Certain edge cases within the AI Dungeon scripting architecture lack definitive documentation and require empirical verification during development:
The maximum character and entry capacity of the storyCards collection before encountering database commit timeouts remains unstated in official documentation[1]. Production implementations should avoid maintaining more than 200 active cards simultaneously to minimize context evaluation latency[4].
Furthermore, while the Adventure Scripts alpha platform documents private state partitioning for chained scripts, the exact property namespacing mechanics and collision resolution behavior across independently published scripts executing in a shared pipeline remain subject to active platform updates[8].

Completeness Audit

A systematic review confirms that all required technical domains are fully specified:

- The execution lifecycle and source concatenation mechanics of the Library tab are documented[3].
- The data structures, persistence contracts, and JSON boundaries of state, history, storyCards, and info are established[1].
- Non-linear lifecycle mutations (Undo, Retry, Continue, Edit) and their specific state desynchronization hazards have been detailed alongside architectural mitigations[1].
- The operational divergence between legacy context editing and prompt caching (info.useCacheEfficient) has been defined to prevent silent failure[4].
- Execution limits (2000 ms timeout, 16 MB heap memory) and sandbox API restrictions (no DOM, Node, network, or asynchronous timers) are fully documented[1].

Works cited

1. Scripting - AI Dungeon Guidebook, https://help.aidungeon.com/scripting
2. What are Scripts and how do you Install them?, https://help.aidungeon.com/what-are-scripts-and-how-do-you-install-them
3. Scripting Guide for AI Dungeon | PDF - Scribd, https://www.scribd.com/document/985223540/Scripting
4. Intro to AI Dungeon Scripting - Worldsmythe's Den, https://worldsmythe.github.io/blog/intro-to-ai-dungeon-scripting/
5. automatic Story Cards, private NPC thoughts, and plot twists for AI, https://www.reddit.com/r/AIDungeon/comments/1vtt7se/rerelease_script_unspoken_turns_automatic_story/
6. Welcome to the Frontier! : r/AIDungeon - Reddit, https://www.reddit.com/r/AIDungeon/comments/1tjumyu/welcome_to_the_frontier/
7. Disappointed in lack of updates for AID : r/AIDungeon - Reddit, https://www.reddit.com/r/AIDungeon/comments/1sz6lqu/disappointed_in_lack_of_

updates_for_aid/
8. Adventure Scripts is on alpha : r/AIDungeon - Reddit, https://www.reddit.com/r/AIDungeon/comments/1vtziiy/adventure_scripts_is_on_alpha/
9. Changelog - ArcQuill, https://arcquill.com/changelog
10. Introduction - /aidg/ Guides and Resources, https://guide.aidg.club/Resources-And-Guides/Scripts/AuthorsNote/
11. How to Restart in AI Dungeon: Commands & Starting a New Adventure, https://www.balinh.com/2025/09/how-to-restart-in-ai-dungeon-commands.html
12. GitHub - latitudegames/Scripting, https://github.com/latitudegames/Scripting
13. Guides - BetterRepository, https://better-repository.netlify.app/guides
14. Small Thoughts, https://smolthots.neocities.org/
15. Editing Text : r/Voyage - Reddit, https://www.reddit.com/r/Voyage/comments/1vny078/editing_text/
16. Scripts - BetterRepository, https://better-repository.netlify.app/scripts
17. AI Dungeon Script Configuration with Story Cards - Worldsmythe's Den, https://worldsmythe.github.io/blog/script-configuration-with-story-cards/
18. state.message is broken : r/AIDungeon - Reddit, https://www.reddit.com/r/AIDungeon/comments/18fy8yc/statemessage_is_broken/
19. Best Practices : r/AIDungeon - Reddit, https://www.reddit.com/r/AIDungeon/comments/k5op6o/best_practices/
20. GitHub - Disty0/KoboldAI, https://github.com/Disty0/KoboldAI
21. How the New Memory System Works - Latitude.io, https://latitude.io/blog/how-the-new-memory-system-works
22. horizonfps/project-lunar: AI-powered interactive RPG engine with, https://github.com/horizonfps/project-lunar
23. Auto Cards | PDF | Boolean Data Type - Scribd, https://www.scribd.com/document/1012308205/Auto-Cards
24. aidungeon · GitHub Topics, https://github.com/topics/aidungeon?l=javascript&o=desc&s=forks
25. Hashtag-DnD - Scenario script for AI Dungeon - GitHub, https://github.com/raeleus/Hashtag-DnD
26. Tutorial: How to actually stop the AI from repeating itself : r/AIDungeon, https://www.reddit.com/r/AIDungeon/comments/1k2etct/tutorial_how_to_actually_stop_the_ai_from/
27. LewdLeah - GitHub, https://github.com/LewdLeah
28. Yi1i1i - GitHub, https://github.com/Yi1i1i
29. AI Dungeon - Grokipedia, https://grokipedia.com/page/AI_Dungeon